

flotationFoam

Annotated line-level differences between the OF-13 (OpenFOAM Foundation) original source and the ported ESI OpenFOAM v2606 (tom-v2606) source

This report compares every .C / .H file in the flotationFoam library between the original OpenFOAM Foundation (OF-13) source (flotationFoam-master/flotationFoam-master/src/flotation) and the ported ESI OpenFOAM v2606 source (tom-v2606/src/flotation), and explains *why* each differing line changed.

Method: files were matched by relative path (one pair was matched by role instead -- see the architecture note below). Every line was normalized (leading/trailing whitespace stripped, internal whitespace runs collapsed) before comparison, so pure formatting differences are not reported -- only lines whose actual content differs are shown, exactly as they appear in each source file.

Annotations: most differences fall into a small number of recurring patterns (an API rename, an idiom change, a genuine bug). Each pattern is explained in full the first time it appears, in reading order (top to bottom through this report); every later occurrence carries a short cross-reference back to that first explanation instead of repeating it, with a clickable link.

File pairs compared	78
Identical (content-wise)	46
Files with real differences	32
Total differing lines (both sides)	451
Distinct recurring patterns identified	14
OF-13 files with no v2606 counterpart	2

Summary of all compared files

File (OF-13 path)	Status	Lines
SherwoodNumbers/RanzMarshallSherwoodNumber/RanzMarshallSherwoodNumber.C	Changed	2
SherwoodNumbers/RanzMarshallSherwoodNumber/RanzMarshallSherwoodNumber.H	Identical	-
SherwoodNumbers/SherwoodNumber/SherwoodNumber.C	Changed	7
SherwoodNumbers/SherwoodNumber/SherwoodNumber.H	Identical	-
SherwoodNumbers/StokesFlowSherwoodNumber/StokesFlowSherwoodNumber.C	Changed	2
SherwoodNumbers/StokesFlowSherwoodNumber/StokesFlowSherwoodNumber.H	Identical	-
SherwoodNumbers/constantSherwoodNumber/constantSherwoodNumber.C	Changed	2
SherwoodNumbers/constantSherwoodNumber/constantSherwoodNumber.H	Identical	-
SherwoodNumbers/potentialFlowSherwoodNumber/potentialFlowSherwoodNumber.C	Changed	2
SherwoodNumbers/potentialFlowSherwoodNumber/potentialFlowSherwoodNumber.H	Identical	-
flotationSystem.C	Changed	75
flotationSystem.H	Changed	11
fvModels/flotation/flotation.C	Renamed + changed	74
fvModels/flotation/flotation.H	Renamed + changed	70
particleDiffusivityModels/StokesEinstein/StokesEinstein.C	Changed	2
particleDiffusivityModels/StokesEinstein/StokesEinstein.H	Identical	-
particleDiffusivityModels/constantDiffusivity/constantDiffusivity.C	Identical	-

File (OF-13 path)	Status	Lines
particleDiffusivityModels/constantDiffusivity/constantDiffusivity.H	Identical	-
particleDiffusivityModels/particleDiffusivityModel/particleDiffusivityModel.C	Changed	9
particleDiffusivityModels/particleDiffusivityModel/particleDiffusivityModel.H	Identical	-
particleModel/particleModel.C	Changed	29
particleModel/particleModel.H	Changed	7
particleSection/particleSection/particleSection.C	Changed	20
particleSection/particleSection/particleSection.H	Identical	-
particleSizeDistributions/logarithmic/logarithmic.C	Identical	-
particleSizeDistributions/logarithmic/logarithmic.H	Identical	-
particleSizeDistributions/monoDisperse/monoDisperse.C	Identical	-
particleSizeDistributions/monoDisperse/monoDisperse.H	Identical	-
particleSizeDistributions/particleSizeDistribution/particleSizeDistribution.C	Changed	11
particleSizeDistributions/particleSizeDistribution/particleSizeDistribution.H	Identical	-
particleSizeDistributions/sectionalList/sectionalList.C	Identical	-
particleSizeDistributions/sectionalList/sectionalList.H	Identical	-
particleTransferModels/continuousBrownianEntrainment/continuousBrownianEntrainment.C	Changed	2
particleTransferModels/continuousBrownianEntrainment/continuousBrownianEntrainment.H	Identical	-
particleTransferModels/continuousParticleTransferModel/continuousParticleTransferModel.C	Changed	12
particleTransferModels/continuousParticleTransferModel/continuousParticleTransferModel.H	Changed	6
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainment.C	Identical	-
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainment.H	Identical	-
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainmentRates/BloomHeindelRate/BloomHeindelRate.C	Identical	-
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainmentRates/BloomHeindelRate/BloomHeindelRate.H	Identical	-
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainmentRates/continuousTurbulentEntrainmentRate/continuousTurbulentEntrainmentRate.C	Changed	7
particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainmentRates/continuousTurbulentEntrainmentRate/continuousTurbulentEntrainmentRate.H	Identical	-
particleTransferModels/dispersedBrownianDeposition/dispersedBrownianDeposition.C	Changed	2
particleTransferModels/dispersedBrownianDeposition/dispersedBrownianDeposition.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterception.C	Identical	-
particleTransferModels/dispersedInterception/dispersedInterception.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/SutherlandEfficiency/SutherlandEfficiency.C	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/SutherlandEfficiency/SutherlandEfficiency.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/YoonLuttrellIntermediateEfficiency/YoonLuttrellIntermediateEfficiency.C	Changed	9
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/YoonLuttrellIntermediateEfficiency/YoonLuttrellIntermediateEfficiency.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/YoonLuttrellStokesEfficiency/YoonLuttrellStokesEfficiency.C	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/YoonLuttrellStokesEfficiency/YoonLuttrellStokesEfficiency.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/dispersedInterceptionEfficiency/dispersedInterceptionEfficiency.C	Changed	24
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/dispersedInterceptionEfficiency/dispersedInterceptionEfficiency.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/idealEfficiency/idealEfficiency.C	Changed	2

File (OF-13 path)	Status	Lines
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/idealEfficiency/idealEfficiency.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/DaiInductionTime/DaiInductionTime.C	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/DaiInductionTime/DaiInductionTime.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/constantInductionTime/constantInductionTime.C	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/constantInductionTime/constantInductionTime.H	Identical	-
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/inductionTime/inductionTime.C	Changed	7
particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/inductionTime/inductionTime.H	Identical	-
particleTransferModels/dispersedParticleTransferModel/dispersedParticleTransferModel.C	Changed	17
particleTransferModels/dispersedParticleTransferModel/dispersedParticleTransferModel.H	Changed	6
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDeposition.C	Identical	-
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDeposition.H	Changed	2
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/SaffmanTurnerRate/SaffmanTurnerRate.C	Changed	3
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/SaffmanTurnerRate/SaffmanTurnerRate.H	Identical	-
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/dispersedTurbulentDepositionRate/dispersedTurbulentDepositionRate.C	Changed	7
particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/dispersedTurbulentDepositionRate/dispersedTurbulentDepositionRate.H	Identical	-
particleTransferModels/particleTransferModel/particleTransferModel.C	Changed	7
particleTransferModels/particleTransferModel/particleTransferModel.H	Identical	-
particleVelocityModels/localEquilibrium/localEquilibrium.C	Changed	6
particleVelocityModels/localEquilibrium/localEquilibrium.H	Identical	-
particleVelocityModels/particleVelocityModel/particleVelocityModel.C	Changed	9
particleVelocityModels/particleVelocityModel/particleVelocityModel.H	Identical	-
particleVelocityModels/passiveTracer/passiveTracer.C	Identical	-
particleVelocityModels/passiveTracer/passiveTracer.H	Identical	-
functionObjects/particleVelocity/particleVelocity.C	Not ported	-
functionObjects/particleVelocity/particleVelocity.H	Not ported	-

Not ported: functionObjects/particleVelocity

`functionObjects/particleVelocity/particleVelocity.{C,H}` exists only in OF-13; there is no v2606 counterpart at all, and no plausible rename target elsewhere in the port.

What it does. `particleVelocity` is an OpenFOAM *function object*

(`Foam::functionObjects::particleVelocity`, derived from `fieldExpression`, OpenFOAM's base class for "compute one field from the registry and store/write it each call"). It is **not** part of the N-transport solve itself -- it is a pure post-processing add-on, activated the same way any function object is (via `system/functions` or `#includeFunc`), configured with its own tiny dictionary naming which phase and section to report on.

How it connects to the rest of the library. On each call, it looks up the currently-running `flotationSystem` object by name (default `flotationProperties`, the same registry name `flotation/flotationSource` constructs it under), picks whichever of the two phases' `particleModel(model1()/model2())` matches the requested phase name, indexes into that phase's `particleSection` list at the requested section number, and calls that section's own `V()` accessor -- which is `fvc::reconstruct(phi())`, turning the section's internal face-flux field (`phi:section`, otherwise a private book-keeping quantity never written to disk) into a proper cell-centred velocity vector field. It stores that field in the registry under a name like `V.water:3`, so it is written to disk at every write time exactly like any other field. In short: it exposes, purely for visualisation, the local drift/settling velocity that a *specific particle size class* in a *specific phase* is moving at -- something the solver already computes internally every timestep (via each particle section's velocity model) but does not otherwise persist anywhere.

Why it wasn't ported. It has zero effect on the solve: the N-transport equations, the attachment/detachment source terms, and every result this project has validated are computed identically whether or not this function object is active -- it only adds an extra *output* field, never read back by anything else. Porting it would mean also adapting it to ESI's own function-object registration conventions (mechanically similar, since ESI's `fieldExpression` base class plays the same role, but not exercised in this project) for a feature that was never needed to validate that the two codes' physics agree, so it was left out of scope. It is flagged here for completeness rather than left silently unmentioned: if per-size-class particle velocity fields are ever wanted for visualisation on the v2606 side, this is the exact OF-13 functionality that would need to be ported to get them, and `particleSection::V()` -- present and unchanged on the v2606 side, since `particleSection.H` is not among the files with real differences -- is the exact hook such a port would call.

Detailed line differences

Only files with real content differences are shown below (identical files are omitted; see the summary table for the full list). Removed lines (OF-13 only) are red with a '-' prefix; added lines (v2606 only) are green with a '+' prefix. Each differing block is followed by a short note: a full explanation the first time a pattern appears, or a cross-reference back to it after that.

SherwoodNumbers/RanzMarshallSherwoodNumber/RanzMarshallSherwoodNumber.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 48 / v2606 line 48 @@

```
- mesh_.time().time().name(),  
+ mesh_.time().timeName(),
```

Why (Restart-breaking Time accessor): OF-13's source builds each per-timestep IOobject with `<mesh>.time().time().name()`. In OF-13's class hierarchy this resolves correctly to the current time-directory name (e.g. "10"). Copied verbatim into the ESI port, the identical-looking expression resolves to something different: ESI's `Time::name()` (reached through the extra `.time()` hop) returns the *case directory's own name* instead of the current time value. The effect is silent: at a true `t=0` start the code falls back to reading the field named by the un-suffixed default (e.g. `N.water`) rather than the section-specific one it actually wants (e.g. `N.water:1`), and since the un-suffixed default at `t=0` is the correct initial condition, nothing looks wrong. On a restart, the same fallback re-reads that same `t=0` initial condition instead of the real, already-evolved field -- silently discarding everything simulated so far. This is a genuine, confirmed bug (found from a real restart discrepancy during this project's validation work, diagnosed down to this exact line, and fixed) -- not just a cosmetic API rename. The fix is `<mesh>.time().timeName()`, OpenFOAM's actual accessor for the current time-directory name.

SherwoodNumbers/SherwoodNumber/SherwoodNumber.C

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

@@ OF-13 line 39 / v2606 line 39 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(type);  
+ auto* ctorPtr = dictionaryConstructorTable(type);
```

@@ OF-13 line 42 / v2606 line 41 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 55 / v2606 line 54 @@

```
- return cstrIter()(mesh, type);  
+ return ctorPtr(mesh, type);
```

Why (Runtime-selection-table lookup idiom): OF-13's older-style runtime-selection lookup fetches an iterator into the constructor hash table, compares it against `end()`, and calls through `cstrIter()`. ESI v2606's version of the same macro-generated table exposes a direct pointer-returning operator `()`, so the equivalent lookup is `auto* ctorPtr = Table(type); if (!ctorPtr) ...; return ctorPtr(...)`; This is a mechanical difference between the two forks' own `declareRunTimeSelectionTable` implementations -- both express exactly the same "look up the constructor function for this type name, error if not found" logic, just through each fork's own idiom; nothing about the port's own design is involved.

SherwoodNumbers/StokesFlowSherwoodNumber/StokesFlowSherwoodNumber.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 53 / v2606 line 53 @@

```
- mesh_.time().time().name(),  
+ mesh_.time().timeName(),
```

Why: same as [Restart-breaking Time accessor](#), explained at its first occurrence above.

SherwoodNumbers/constantSherwoodNumber/constantSherwoodNumber.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 65 / v2606 line 65 @@

```
- mesh_.time().time().name(),  
+ mesh_.time().timeName(),
```

Why: same as [Restart-breaking Time accessor](#), explained at its first occurrence above.

SherwoodNumbers/potentialFlowSherwoodNumber/potentialFlowSherwoodNumber.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 56 / v2606 line 56 @@

```
- mesh_.time().time().name(),  
+ mesh_.time().timeName(),
```

Why: same as [Restart-breaking Time accessor](#), explained at its first occurrence above.

flotationSystem.C

9 differing block(s), 75 lines touched (whitespace-only differences excluded)

@@ OF-13 line 4 / v2606 line 4 @@

```
- #include "fvModels.H"  
- #include "fvConstraints.H"  
+ #include "turbulenceModel.H"
```

Note: Swaps the fvModels.H/fvConstraints.H includes for turbulenceModel.H -- this file doesn't solve equations itself (that's particleModel.C's job), so the OF-13 includes were unused here in the first place; the new include is needed for the turbulence-model rename used below.

@@ OF-13 line 12 / v2606 line 11 @@

```
+ }  
+  
+ // * * * * * Local Functions * * * * *  
+  
+ namespace  
+ {  
+ // Split an interface name of the form "<phase1>_<phase2>" (e.g.  
+ // "water_air") into references to its two constituent phaseModels,  
+ // by checking every ordered pair of the phaseSystem's actual phases -  
+ // this is the ESI-side replacement for Foundation's phaseInterface  
+ // word-parsing constructor, which ESI's phasePair doesn't have (it  
+ // always takes two phaseModel references directly).  
+ void splitInterfaceName  
+ (  
+ const Foam::phaseSystem& fluid,  
+ const Foam::word& interfaceName,  
+ const Foam::phaseModel*& phase1,  
+ const Foam::phaseModel*& phase2  
+ )  
+ {  
+ forAll(fluid.phases(), i)  
+ {  
+ forAll(fluid.phases(), j)  
+ {  
+ if (i == j)  
+ {  
+ continue;  
+ }  
+  
+ const Foam::word& name1 = fluid.phases()[i].name();  
+ const Foam::word& name2 = fluid.phases()[j].name();  
+  
+ if (name1 + "_" + name2 == interfaceName)  
+ {  
+ phase1 = &fluid.phases()[i];  
+ phase2 = &fluid.phases()[j];  
+ return;  
+ }  
+ }  
+ }  
+  
+ FatalErrorInFunction  
+ << "Interface name " << interfaceName << " could not be split "  
+ << "into two known phases of the form <phase1>_<phase2>."  
+ << Foam::exit(Foam::FatalError);  
+ }  
+  
+ Foam::phasePair makeInterfacePair  
+ (  
+ const Foam::phaseSystem& fluid,  
+ const Foam::word& interfaceName
```

```
+ )
+ {
+ const Foam::phaseModel* phase1 = nullptr;
+ const Foam::phaseModel* phase2 = nullptr;
+ splitInterfaceName(fluid, interfaceName, phase1, phase2);
+ return Foam::phasePair(*phase1, *phase2);
+ }
```

@@ OF-13 line 45 / v2606 line 102 @@

```
- interface_(fluid, lookup("interface")),
+ interface_(makeInterfacePair(fluid, word(lookup("interface")))),
```

Why (N-phase interface classes vs ESI's phase-pair classes): OF-13's phase-pair bookkeeping is built on a generic, N-phase-capable phaseInterface class hierarchy, refined into ordered variants such as dispersedPhaseInterface for the transfer models. It can be constructed directly from a phase-pair *name* (e.g. "water_air"), because it knows how to search the whole N-phase system for the two phases that name refers to. ESI v2606's equivalent, phasePair (and its ordered variant orderedPhasePair), is built for its simpler two-phase-at-a-time interfacial-model framework and only constructs from two phaseModel references directly -- it has no name-parsing constructor. Since flotationProperties still specifies the interface by name (interface water_air;), the port adds a small local helper (splitInterfaceName / makeInterfacePair in flotationSystem.C) that does what OF-13's constructor did internally: scan every ordered pair of the phase system's actual phases for the one matching the given name, then build the ESI pair type from the two phaseModel references it finds. Occurrences of dispersedPhaseInterface -> orderedPhasePair elsewhere in the transfer-model classes are the same substitution, for the dispersed/continuous-specific variant of the same class family rather than the generic base.

@@ OF-13 line 131 / v2606 line 188 @@

```
- const Foam::phaseCompressible::momentumTransportModel&
+ const Foam::phaseCompressibleTurbulenceModel&
```

@@ OF-13 line 135 / v2606 line 192 @@

```
- mesh_.lookupObject<phaseCompressible::momentumTransportModel>
+ mesh_.lookupObject<phaseCompressibleTurbulenceModel>
```

@@ OF-13 line 139 / v2606 line 196 @@

```
- momentumTransportModel::typeName,
+ turbulenceModel::propertiesName,
```

@@ OF-13 line 145 / v2606 line 202 @@

```
- const Foam::phaseCompressible::momentumTransportModel&
+ const Foam::phaseCompressibleTurbulenceModel&
```

@@ OF-13 line 149 / v2606 line 206 @@

```
- mesh_.lookupObject<phaseCompressible::momentumTransportModel>
+ mesh_.lookupObject<phaseCompressibleTurbulenceModel>
```

@@ OF-13 line 153 / v2606 line 210 @@

```
- momentumTransportModel::typeName,
+ turbulenceModel::propertiesName,
```

Why (Turbulence-model class rename): OF-13 (Foundation, current releases) renamed its turbulence-modelling framework from turbulenceModel to momentumTransportModel (part of a broader generalisation to cover momentum transport generically, not just Reynolds-stress closures), so the phase-specific class became Foam::phaseCompressible::momentumTransportModel and its registered-field name is momentumTransportModel::typeName. ESI v2606 has not adopted this rename and still calls the equivalent class phaseCompressibleTurbulenceModel, exposing the field's registered name as turbulenceModel::propertiesName. The port substitutes the ESI-side name and header wherever OF-13 looks up or declares this class; the underlying object (whichever turbulence model -- laminar, k-epsilon, etc. -- was selected in the phase's turbulence properties) is the same, only the C++ type name and header differ.

flotationSystem.H

5 differing block(s), 11 lines touched (whitespace-only differences excluded)

@@ OF-13 line 6 / v2606 line 6 @@

```
- #include "phaseInterface.H"
+ #include "phasePair.H"
+ #include "phaseCompressibleTurbulenceModel.H"
```

Note: Combines two renames in one include line: the phaseInterface -> phasePair rename (see the interface-classes explanation in flotationSystem.C above) and the turbulence-model rename (also explained there).

@@ OF-13 line 38 / v2606 line 39 @@

```
- phaseInterface interface_;  
+ phasePair interface_;
```

@@ OF-13 line 100 / v2606 line 101 @@

```
- inline const phaseInterface& interface() const  
+ inline const phasePair& interface() const
```

Why: same as ***N-phase interface classes vs ESI's phase-pair classes***, explained at its first occurrence above.

@@ OF-13 line 150 / v2606 line 151 @@

```
- const phaseCompressible::momentumTransportModel&  
+ const phaseCompressibleTurbulenceModel&
```

@@ OF-13 line 154 / v2606 line 155 @@

```
- const phaseCompressible::momentumTransportModel&  
+ const phaseCompressibleTurbulenceModel&
```

Why: same as ***Turbulence-model class rename***, explained at its first occurrence above.

fvModels/flotation/flotation.C

17 differing block(s), 74 lines touched (whitespace-only differences excluded)

fvModels/flotation/flotation.H

12 differing block(s), 70 lines touched (whitespace-only differences excluded)

particleDiffusivityModels/StokesEinstein/StokesEinstein.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 47 / v2606 line 47 @@

```
- const volScalarField mu(model_.phase().fluidThermo().mu());  
+ const volScalarField mu(model_.phase().thermo().mu());
```

Why (Phase material-property accessor rename): OF-13 exposes a phase's material-property model (density, viscosity, etc.) through `phaseModel::fluidThermo()`. ESI v2606's `phaseModel` exposes the same functionality through a shorter, more generic accessor, `phaseModel::thermo()` (ESI's phase-thermo hierarchy isn't named around fluids specifically). Every occurrence of this pattern is the same rename: whichever transport property is being read (`.nu()`, `.mu()`, ...), only the accessor name changes -- the underlying property values, and the `thermophysicalProperties` dictionary they come from, are identical.

particleDiffusivityModels/particleDiffusivityModel/particleDiffusivityModel.C

4 differing block(s), 9 lines touched (whitespace-only differences excluded)

@@ OF-13 line 26 / v2606 line 26 @@

```
- model.system().mesh().time().time().name(),  
+ model.system().mesh().time().timeName(),
```

Why: same as ***Restart-breaking Time accessor***, explained at its first occurrence above.

@@ OF-13 line 53 / v2606 line 53 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(particleDiffusivityModelType);  
+ auto* ctorPtr = dictionaryConstructorTable(particleDiffusivityModelType);
```

@@ OF-13 line 56 / v2606 line 55 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 66 / v2606 line 65 @@

```
- return cstrIter()(model, dict);  
+ return ctorPtr(model, dict);
```

Why: same as ***Runtime-selection-table lookup idiom***, explained at its first occurrence above.

particleModel/particleModel.C

11 differing block(s), 29 lines touched (whitespace-only differences excluded)

@@ OF-13 line 5 / v2606 line 5 @@

```
- #include "fvModels.H"  
- #include "fvConstraints.H"  
+ #include "fvOptions.H"
```



```
+ #include "turbulenceModel.H"
```

Why (fvModels + fvConstraints unified into fv::options): OF-13 (Foundation) splits generic per-timestep customisation into two separate global registries: fvModels (explicit source terms, fvModels.source(field)) and fvConstraints (post-solve bounding/constraining, fvConstraints.constrain(...)). ESI v2606 has never split these apart -- both roles live in one fv::options object, exposing the source-term contribution as fvOptions(field), the equation-constraining hook as fvOptions.constrain(matrix), and a post-solve field-correction hook as fvOptions.correct(field). The port routes both of OF-13's calls through the one ESI object; note that OF-13's post-solve fvConstraints.constrain(N) becomes ESI's fvOptions.correct(N) rather than a second constrain() call, since ESI's constrain() only targets the assembled matrix (used once, above, for the equation itself) and correct() is the separate hook for adjusting the already-solved field afterwards. This same unification is the underlying reason flotationSource is built as a single ESI fvOption rather than as OF-13's separate fvModel -- see the architecture note further up.

```
@@ OF-13 line 86 / v2606 line 86 @@
```

```
- const fvModels& fvModels(fvModels::New(system_.mesh()));  
-  
- const fvConstraints& fvConstraints  
- (  
-   fvConstraints::New(system_.mesh())  
- );  
+ fv::options& fvOptions(fv::options::New(system_.mesh()));
```

```
@@ OF-13 line 132 / v2606 line 127 @@
```

```
- + fvModels.source(N)  
+ + fvOptions(N)
```

```
@@ OF-13 line 135 / v2606 line 130 @@
```

```
- fvConstraints.constrain(NEqn);  
+ fvOptions.constrain(NEqn);
```

```
@@ OF-13 line 149 / v2606 line 144 @@
```

```
- fvConstraints.constrain(N);  
+ fvOptions.correct(N);
```

Why: same as **fvModels + fvConstraints unified into fv::options**, explained at its first occurrence above.

```
@@ OF-13 line 155 / v2606 line 150 @@
```

```
- const Foam::phaseCompressible::momentumTransportModel&  
+ const Foam::phaseCompressibleTurbulenceModel&
```

```
@@ OF-13 line 159 / v2606 line 154 @@
```

```
- system_.mesh().lookupObject<phaseCompressible::momentumTransportModel>  
+ system_.mesh().lookupObject<phaseCompressibleTurbulenceModel>
```

```
@@ OF-13 line 163 / v2606 line 158 @@
```

```
- momentumTransportModel::typeName,  
+ turbulenceModel::propertiesName,
```

```
@@ OF-13 line 169 / v2606 line 164 @@
```

```
- const Foam::phaseCompressible::momentumTransportModel&  
+ const Foam::phaseCompressibleTurbulenceModel&
```

```
@@ OF-13 line 173 / v2606 line 168 @@
```

```
- system_.mesh().lookupObject<phaseCompressible::momentumTransportModel>  
+ system_.mesh().lookupObject<phaseCompressibleTurbulenceModel>
```

```
@@ OF-13 line 177 / v2606 line 172 @@
```

```
- momentumTransportModel::typeName,  
+ turbulenceModel::propertiesName,
```

Why: same as **Turbulence-model class rename**, explained at its first occurrence above.

particleModel/particleModel.H

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

```
@@ OF-13 line 5 / v2606 line 5 @@
```

```
- #include "phaseCompressibleMomentumTransportModel.H"  
+ #include "phaseModel.H"  
+ #include "phaseCompressibleTurbulenceModel.H"
```

```
@@ OF-13 line 122 / v2606 line 123 @@
```

```
- const phaseCompressible::momentumTransportModel& turbulence() const;  
+ const phaseCompressibleTurbulenceModel& turbulence() const;
```

@@ OF-13 line 125 / v2606 line 126 @@

```
- const phaseCompressible::momentumTransportModel&  
+ const phaseCompressibleTurbulenceModel&
```

Why: same as **Turbulence-model class rename**, explained at its first occurrence above.

particleSection/particleSection/particleSection.C

10 differing block(s), 20 lines touched (whitespace-only differences excluded)

@@ OF-13 line 42 / v2606 line 42 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 50 / v2606 line 50 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 64 / v2606 line 64 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 82 / v2606 line 82 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 98 / v2606 line 98 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 127 / v2606 line 127 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 146 / v2606 line 146 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

@@ OF-13 line 165 / v2606 line 165 @@

```
- model_.system().mesh().time().time().name(),  
+ model_.system().mesh().time().timeName(),
```

Why: same as **Restart-breaking Time accessor**, explained at its first occurrence above.

@@ OF-13 line 55 / v2606 line 55 @@

```
- if (fieldHeader.headerOk())  
+ if (fieldHeader.typeHeaderOk<volScalarField>(true))
```

@@ OF-13 line 73 / v2606 line 73 @@

```
- else if (defaultFieldHeader.headerOk())  
+ else if (defaultFieldHeader.typeHeaderOk<volScalarField>(true))
```

Why (Typed vs untyped IObject header check): OF-13's `IObject::headerOk()` only checks that a field header exists and is well-formed, without checking what *type* of field it declares. ESI v2606 deprecated this in favour of the templated `typeHeaderOk<Type>(checkType)`, which additionally verifies the on-disk field is actually a `volScalarField` (here) before the code commits to reading it as one -- a stricter, type-safe version of the same existence check. Both call sites needed this more specific form; the logic itself -- try the section-specific field first, fall back to the single default field -- is unchanged.

particleSizeDistributions/particleSizeDistribution/particleSizeDistribution.C

5 differing block(s), 11 lines touched (whitespace-only differences excluded)

@@ OF-13 line 25 / v2606 line 25 @@

```
- mesh.time().time().name(),  
+ mesh.time().timeName(),
```

@@ OF-13 line 48 / v2606 line 48 @@

```
- mesh.time().time().name(),  
+ mesh.time().timeName(),
```

Why: same as **Restart-breaking Time accessor**, explained at its first occurrence above.

@@ OF-13 line 76 / v2606 line 76 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(particleSizeDistributionType);  
+ auto* ctorPtr = dictionaryConstructorTable(particleSizeDistributionType);
```

@@ OF-13 line 79 / v2606 line 78 @@

```
- if (cstrIter == dictionaryConstructorTablePtr->end())
+ if (!ctorPtr)
```

@@ OF-13 line 89 / v2606 line 88 @@

```
- return cstrIter()(dict, mesh, true);
+ return ctorPtr(dict, mesh, true);
```

Why: same as [Runtime-selection-table lookup idiom](#), explained at its first occurrence above.

particleTransferModels/continuousBrownianEntrainment/continuousBrownianEntrainment.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 56 / v2606 line 56 @@

```
- const volScalarField nu(interface_.dispersed().fluidThermo().nu());
+ const volScalarField nu(interface_.dispersed().thermo().nu());
```

Why: same as [Phase material-property accessor rename](#), explained at its first occurrence above.

particleTransferModels/continuousParticleTransferModel/continuousParticleTransferModel.C

2 differing block(s), 12 lines touched (whitespace-only differences excluded)

@@ OF-13 line 31 / v2606 line 31 @@

```
- interface_
+ wordList
+ {
+ interface_.phase1().name(),
+ interface_.phase2().name()
+ }
```

Why (Explicit phase-name list for blending lookup): The interfacial-model list that keys transfer-model coefficients by phase-pair name needs an explicit two-word phase-name list to register itself against. OF-13's generic phaseInterface-family object can supply this list itself (it already knows, and can enumerate, its own constituent phases generically). ESI's orderedPhasePair doesn't expose an equivalent generic accessor, so the port builds the name list explicitly from interface_.phase1().name() and interface_.phase2().name() wherever OF-13 passed the interface object directly.

@@ OF-13 line 49 / v2606 line 53 @@

```
- blending_->f1DispersedIn2
- (
- interface_.fluid().phases()
- .PtrList<phaseModel>::convert<const volScalarField>()
- )
+ blending_->f1(interface_.dispersed(), interface_.continuous())
```

Why (Blending-method call signature): OF-13's generic, N-phase blending framework computes a blending factor for one phase via blending_->f1DispersedIn2(alphas), taking the full list of every phase's volume-fraction field in the system, so it can blend correctly regardless of how many phases are present. ESI v2606's blending framework is built for exactly two phases at a time, so the equivalent call is the simpler blending_->f1(dispersed, continuous), taking the two phase models directly. Both compute the same physical blending factor (based on how locally continuous/dispersed each phase is); only the calling convention differs, following each fork's native interfacial-model API -- the same architecture difference already identified and validated elsewhere in this project's drag/lift model comparisons.

particleTransferModels/continuousParticleTransferModel/continuousParticleTransferModel.H

3 differing block(s), 6 lines touched (whitespace-only differences excluded)

@@ OF-13 line 5 / v2606 line 5 @@

```
- #include "dispersedPhaseInterface.H"
+ #include "orderedPhasePair.H"
```

@@ OF-13 line 29 / v2606 line 29 @@

```
- dispersedPhaseInterface interface_;
+ orderedPhasePair interface_;
```

@@ OF-13 line 53 / v2606 line 53 @@

```
- inline const dispersedPhaseInterface& interface() const
+ inline const orderedPhasePair& interface() const
```

Why: same as [N-phase interface classes vs ESI's phase-pair classes](#), explained at its first occurrence above.

particleTransferModels/continuousTurbulentEntrainment/continuousTurbulentEntrainmentRates/continuousTurbulentEntrainmentRate/continuousTurbulentEntrainmentRate.C

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

@@ OF-13 line 44 / v2606 line 44 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(modelType);  
+ auto* ctorPtr = dictionaryConstructorTable(modelType);
```

@@ OF-13 line 47 / v2606 line 46 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 56 / v2606 line 55 @@

```
- return cstrIter()(transfer, dict);  
+ return ctorPtr(transfer, dict);
```

Why: same as [Runtime-selection-table lookup idiom](#), explained at its first occurrence above.

particleTransferModels/dispersedBrownianDeposition/dispersedBrownianDeposition.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 53 / v2606 line 53 @@

```
- const volScalarField nu(interface_.continuous().fluidThermo().nu());  
+ const volScalarField nu(interface_.continuous().thermo().nu());
```

Why: same as [Phase material-property accessor rename](#), explained at its first occurrence above.

particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/YoonLuttrellIntermediateEfficiency/YoonLuttrellIntermediateEfficiency.C

1 differing block(s), 9 lines touched (whitespace-only differences excluded)

@@ OF-13 line 51 / v2606 line 51 @@

```
- const volScalarField d(transfer_.interface().dispersed().d());  
+ const volScalarField d  
+ (  
+ max  
+ (  
+ transfer_.interface().dispersed().d(),  
+ dimensionedScalar(dimLength, SMALL)  
+ )  
+ );
```

Why (New defensive zero-diameter guard (v2606 only)): v2606 adds a small defensive guard here with no OF-13 counterpart: wherever a particle-section diameter `d()` is used as a divisor (interception efficiency, transfer-rate-per-particle-volume), the port clamps it with `max(d(), dimensionedScalar(dimLength, SMALL))` before dividing. OF-13 divides by `d()` directly. Since the particle-size distribution used throughout this project is fixed at construction (a sectionalList spanning 1 nm - 100 µm) and never reaches exactly zero in practice, this guard is inert in every case this project ran -- it only changes behaviour if a diameter field were ever driven to zero, which the validated flotationProperties configuration never does.

particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/dispersedInterceptionEfficiency/dispersedInterceptionEfficiency.C

4 differing block(s), 24 lines touched (whitespace-only differences excluded)

@@ OF-13 line 67 / v2606 line 67 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find  
- (  
- dispersedInterceptionEfficiencyType  
- );  
+ auto* ctorPtr =  
+ dictionaryConstructorTable(dispersedInterceptionEfficiencyType);
```

@@ OF-13 line 73 / v2606 line 70 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 83 / v2606 line 80 @@

```
- return cstrIter()(transfer, dict);  
+ return ctorPtr(transfer, dict);
```

Why: same as [Runtime-selection-table lookup idiom](#), explained at its first occurrence above.

@@ OF-13 line 107 / v2606 line 104 @@

```
- return  
- transfer_.model().system().distribution()[i]  
- / transfer_.interface().dispersed().d();  
+ const volScalarField dSafe  
+ (  
+ max  
+ (  
+ transfer_.interface().dispersed().d(),  
+ dimensionedScalar(dimLength, SMALL)  
+ )  
+ );  
+ return transfer_.model().system().distribution()[i]/dSafe;
```

Why: same as [New defensive zero-diameter guard \(v2606 only\)](#), explained at its first occurrence above.

particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/idealEfficiency/idealEfficiency.C

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 58 / v2606 line 58 @@

```
- transfer_.model().system().mesh().time().time().name(),  
+ transfer_.model().system().mesh().time().timeName(),
```

Why: same as [Restart-breaking Time accessor](#), explained at its first occurrence above.

particleTransferModels/dispersedInterception/dispersedInterceptionEfficiencies/inductionTimes/inductionTime/inductionTime.C

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

@@ OF-13 line 44 / v2606 line 44 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(inductionTimeType);  
+ auto* ctorPtr = dictionaryConstructorTable(inductionTimeType);
```

@@ OF-13 line 47 / v2606 line 46 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 56 / v2606 line 55 @@

```
- return cstrIter()(transfer, dict);  
+ return ctorPtr(transfer, dict);
```

Why: same as [Runtime-selection-table lookup idiom](#), explained at its first occurrence above.

particleTransferModels/dispersedParticleTransferModel/dispersedParticleTransferModel.C

4 differing block(s), 17 lines touched (whitespace-only differences excluded)

@@ OF-13 line 31 / v2606 line 31 @@

```
- interface_  
+ wordList  
+ {  
+ interface_.phase1().name(),  
+ interface_.phase2().name()  
+ }
```

Why: same as [Explicit phase-name list for blending lookup](#), explained at its first occurrence above.

@@ OF-13 line 51 / v2606 line 55 @@

```
- blending_->f1DispersedIn2  
+ blending_->f1(interface_.dispersed(), interface_.continuous())  
+ );  
+ const volScalarField dSafe
```

Why: same as [Blending-method call signature](#), explained at its first occurrence above.

Note: This hunk also introduces the `dSafe` local used just below -- see the defensive zero-diameter guard explanation further down this file.

@@ OF-13 line 53 / v2606 line 60 @@

```
- interface_.fluid().phases()
- .PtrList<phaseModel>::convert<const volScalarField>()
- )
+ max(interface_.dispersed().d(), dimensionedScalar(dimLength, SMALL))
```

@@ OF-13 line 64 / v2606 line 69 @@

```
- / (pi*pow3(interface_.dispersed().d())),
+ / (pi*pow3(dSafe)),
```

Why: same as [New defensive zero-diameter guard \(v2606 only\)](#), explained at its first occurrence above.

particleTransferModels/dispersedParticleTransferModel/dispersedParticleTransferModel.H

3 differing block(s), 6 lines touched (whitespace-only differences excluded)

@@ OF-13 line 5 / v2606 line 5 @@

```
- #include "dispersedPhaseInterface.H"
+ #include "orderedPhasePair.H"
```

@@ OF-13 line 29 / v2606 line 29 @@

```
- dispersedPhaseInterface interface_;
+ orderedPhasePair interface_;
```

@@ OF-13 line 53 / v2606 line 53 @@

```
- inline const dispersedPhaseInterface& interface() const
+ inline const orderedPhasePair& interface() const
```

Why: same as [N-phase interface classes vs ESI's phase-pair classes](#), explained at its first occurrence above.

particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDeposition.H

1 differing block(s), 2 lines touched (whitespace-only differences excluded)

@@ OF-13 line 5 / v2606 line 5 @@

```
- #include "dispersedPhaseInterface.H"
+ #include "orderedPhasePair.H"
```

Why: same as [N-phase interface classes vs ESI's phase-pair classes](#), explained at its first occurrence above.

particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/SaffmanTurnerRate/SaffmanTurnerRate.C

1 differing block(s), 3 lines touched (whitespace-only differences excluded)

@@ OF-13 line 62 / v2606 line 62 @@

```
- const volScalarField& nu =
- transfer_.interface().continuous().fluidThermo().nu();
+ const volScalarField nu(transfer_.interface().continuous().thermo().nu());
```

Why: same as [Phase material-property accessor rename](#), explained at its first occurrence above.

particleTransferModels/dispersedTurbulentDeposition/dispersedTurbulentDepositionRates/dispersedTurbulentDepositionRate/dispersedTurbulentDepositionRate.C

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

@@ OF-13 line 45 / v2606 line 45 @@

```
- dictionaryConstructorTable::iterator cstrIter =
- dictionaryConstructorTablePtr_->find(modelType);
+ auto* ctorPtr = dictionaryConstructorTable(modelType);
```

@@ OF-13 line 48 / v2606 line 47 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())
+ if (!ctorPtr)
```

@@ OF-13 line 57 / v2606 line 56 @@

```
- return cstrIter()(transfer, dict);
+ return ctorPtr(transfer, dict);
```

Why: same as [Runtime-selection-table lookup idiom](#), explained at its first occurrence above.

particleTransferModels/particleTransferModel/particleTransferModel.C

3 differing block(s), 7 lines touched (whitespace-only differences excluded)

@@ OF-13 line 21 / v2606 line 21 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(type);  
+ auto* ctorPtr = dictionaryConstructorTable(type);
```

@@ OF-13 line 24 / v2606 line 23 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 34 / v2606 line 33 @@

```
- return autoPtr<particleTransferModel>(cstrIter()(model, dict));  
+ return autoPtr<particleTransferModel>(ctorPtr(model, dict));
```

Why: same as **Runtime-selection-table lookup idiom**, explained at its first occurrence above.

particleVelocityModels/localEquilibrium/localEquilibrium.C

3 differing block(s), 6 lines touched (whitespace-only differences excluded)

@@ OF-13 line 6 / v2606 line 6 @@

```
- #include "uniformDimensionedFields.H"  
+ #include "gravityMeshObject.H"
```

@@ OF-13 line 48 / v2606 line 48 @@

```
- model_.system().mesh().lookupObject<uniformDimensionedVectorField>("g")  
+ meshObjects::gravity::New(model_.system().mesh().time())
```

Why (Gravity field lookup vs mesh-object singleton): OF-13 reads gravity as a plain registered field, looked up by name: `mesh.lookupObject<uniformDimensionedVectorField>("g")`. ESI v2606 has moved gravity into its generic mesh-object framework -- a lazily-constructed, mesh-scoped singleton, accessed as `meshObjects::gravity::New(mesh.time())`. Both ultimately return the same constant/g value; the port just uses ESI's accessor for it.

@@ OF-13 line 60 / v2606 line 60 @@

```
- model_.phase().fluidThermo().nu()  
+ model_.phase().thermo().nu()
```

Why: same as **Phase material-property accessor rename**, explained at its first occurrence above.

particleVelocityModels/particleVelocityModel/particleVelocityModel.C

4 differing block(s), 9 lines touched (whitespace-only differences excluded)

@@ OF-13 line 26 / v2606 line 26 @@

```
- model.system().mesh().time().time().name(),  
+ model.system().mesh().time().timeName(),
```

Why: same as **Restart-breaking Time accessor**, explained at its first occurrence above.

@@ OF-13 line 53 / v2606 line 53 @@

```
- dictionaryConstructorTable::iterator cstrIter =  
- dictionaryConstructorTablePtr_->find(particleVelocityModelType);  
+ auto* ctorPtr = dictionaryConstructorTable(particleVelocityModelType);
```

@@ OF-13 line 56 / v2606 line 55 @@

```
- if (cstrIter == dictionaryConstructorTablePtr_->end())  
+ if (!ctorPtr)
```

@@ OF-13 line 66 / v2606 line 65 @@

```
- return cstrIter()(model, dict);  
+ return ctorPtr(model, dict);
```

Why: same as **Runtime-selection-table lookup idiom**, explained at its first occurrence above.

Architecture note: OF-13 fvModel vs v2606 fvOption

`fvModels/flotation/flotation.{C,H}` → `fvOptions/flotationSource/flotationSource.{C,H}`

What each framework is. OF-13's `fvModel` is a generic, unified mechanism for "anything that modifies the equations being solved" -- momentum sources, energy sources, species sources, or arbitrary custom physics -- all registered under `constant/fvModels`. Every registered `fvModel` gets a `correct()` call, unconditionally, once per outer PIMPLE/SIMPLE iteration, regardless of which fields it actually touches. That is what "field-agnostic" means here: the framework does not know or care which fields a given `fvModel` reads or writes -- it simply calls `correct()` on every registered model, once per outer iteration, and trusts each one to do its own thing (add source terms via `addSup()`, or, as in `flotation`'s case, solve an entirely separate set of equations inside `correct()` itself).

ESI's `fvOption` is built around a different contract: each option declares which named *fields* it applies to (`fieldNames_`), and the solver calls it at specific, field-scoped points -- `fvOptions.addSup(matrix)` when assembling a particular field's equation, `fvOptions.constrain(matrix)` right before solving it, and `fvOptions.correct(field)` right after a *specific* field has been updated. There is no "call this once per iteration, regardless of field" hook anywhere in this framework -- every entry point is tied to a named field.

The consequence for `flotationSource`. `flotation`'s actual job -- solving twelve sectional N-transport equations across two phases -- has nothing to do with any one field's own equation, so there is no natural `fvOption` hook for it. The port's solution, visible directly in the source comment carried into `flotationSource.C`, is to register `flotationSource` against `U.water` (`fieldNames_ = ["U.<phase1>"]`) purely so its `correct(volVectorField& U)` override gets invoked by the solver's ordinary per-field bookkeeping, and to use that invocation purely as a timing signal -- "an iteration just happened" -- to trigger `flotationSystem::solve()`. The N-equations themselves never touch `U.water` at all; that field is chosen only because it is a real field the solver is guaranteed to correct every outer iteration.

Quantified consequence, verified from this project's own solver logs. OF-13's `fvModels.correct()` is called exactly once per PIMPLE outer corrector: in the laminar bubble-column-plus-flotation validation run built earlier in this project, the "`flotationSystem::`" solve banner appears exactly 3 times per timestep, matching that case's `nOuterCorrectors 3` setting. The v2606 port, because it is tied to `correct(U.water)` rather than a true once-per-iteration hook, is invoked 6 times per timestep in the very same case -- *twice* per outer corrector, since the solver legitimately "corrects" `U.water` twice within each outer loop (once during the momentum predictor's own `fvOptions` pass, once again after the pressure correction updates it). In other words, the port solves the same N-transport equations twice as often, per outer iteration, as the original does. This was confirmed directly by counting solve banners in both codes' logs for the identical case, not inferred from reading the source alone.

This does not obviously bias the final answer: each extra call re-solves an equation whose right-hand side has not changed since the previous call within the same outer iteration, so the extra solves are close to idempotent rather than double-counting a source term. But it is a real, structural difference in how often the port's core equations are exercised, and a plausible -- though unconfirmed, and not the dominant factor identified elsewhere in this project's validation work -- contributor to the small remaining differences between the two codes' velocity and particle-count fields.

Beyond this central difference, several smaller changes below are direct, mechanical consequences of the same `fvModel`->`fvOption` rename (constructor argument order, how the options-specific coefficient dictionary is accessed, and which mesh-motion hooks a derived class must implement) -- each is explained once, in place, the first time it appears below, exactly like the other recurring patterns in this report. All other hunks in this file pair -- class name, `TypeName` string, comment headers, and similar -- are pure renaming with no further explanation needed, and are shown without an individual note.

`fvModels/flotation/flotation.C` → `fvOptions/flotationSource/flotationSource.C`

17 differing block(s), 74 lines touched (whitespace-only differences excluded)

@@ OF-13 line 1 / v2606 line 1 @@

```
- #include "flotation.H"
- #include "fvModels.H"
```



```
+ /*-----*\
+ License
+ This file is part of a user library, built against OpenFOAM v2606.
+ \*-----*/
+
+ #include "flotationSource.H"
```

Note: Header/include swap -- consequence of deriving from option instead of fvModel (see the architecture note above).

@@ OF-13 line 11 / v2606 line 15 @@

```
- defineTypeNameAndDebug(flotation, 0);
+ defineTypeNameAndDebug(flotationSource, 0);
+
```

@@ OF-13 line 14 / v2606 line 19 @@

```
- fvModel,
- flotation,
+ option,
+ flotationSource,
```

@@ OF-13 line 21 / v2606 line 26 @@

```
+
```

@@ OF-13 line 23 / v2606 line 29 @@

```
- Foam::fv::flotation::flotation
+ Foam::fv::flotationSource::flotationSource
```

@@ OF-13 line 27 / v2606 line 33 @@

```
- const fvMesh& mesh,
- const dictionary& dict
+ const dictionary& dict,
+ const fvMesh& mesh
```

Why (Constructor argument order): ESI's `fv::option` base class constructor takes (name, modelType, dict, mesh) -- dictionary before mesh -- the reverse of Foundation's `fvModel` base, (name, modelType, mesh, dict). The port matches its own constructor's argument order to whichever base class it derives from.

@@ OF-13 line 31 / v2606 line 37 @@

```
- fvModel(name, modelType, mesh, dict)
+ option(name, modelType, dict, mesh)
```

@@ OF-13 line 36 / v2606 line 42 @@

```
- if (dict.found("dict"))
+ if (coeffs_.found("dict"))
```

Why (Reading via the option's own coeffs_ member): OF-13's `fvModel` constructor gives derived classes the raw dictionary they were constructed from; the port instead reads from `coeffs_`, a protected member ESI's `fv::option` base class already extracts for its derived classes (the sub-dictionary of coefficients specific to this option). Same dictionary content, read through the accessor the base class already provides.

@@ OF-13 line 40 / v2606 line 46 @@

```
- new flotationSystem(fluid, mesh, word(dict.lookup("dict")))
+ new flotationSystem(fluid, mesh, word(coeffs_.lookup("dict")))
```

Why: same as [Reading via the option's own coeffs_ member](#), explained at its first occurrence above.

@@ OF-13 line 45 / v2606 line 51 @@

```
- system_.reset
- (
- new flotationSystem(fluid, mesh)
- );
- }
+ system_.reset(new flotationSystem(fluid, mesh));
```

@@ OF-13 line 52 / v2606 line 54 @@

```
- // * * * * * Destructor * * * * * //
+ // Use the first interface phase's U field just as a once-per-iteration
+ // clock tick to trigger the solve.
+ fieldNames_.setSize
+ (
+ 1,
+ IOObject::groupName("U", system_->interface().phase1().name())
+ );
+ resetApplied();
```

```
+ }
```

Note: Adds the field-registration workaround described in the architecture note above -- this is the concrete code that ties flotationSource's clock-tick to U.water.

```
@@ OF-13 line 54 / v2606 line 64 @@
```

```
- Foam::fv::flotation::~flotation()  
- {}
```

Note: OF-13's explicit empty destructor is dropped; ESI's base class default is used instead (the destructor is defined inline in the header on the v2606 side -- see flotationSource.H below).

```
@@ OF-13 line 59 / v2606 line 67 @@
```

```
- void Foam::fv::flotation::correct()  
+ void Foam::fv::flotationSource::correct(volVectorField& U)
```

Note: The per-field correct(volVectorField& U) signature itself, versus OF-13's field-agnostic correct() -- this is the core of the architecture difference explained above.

```
@@ OF-13 line 61 / v2606 line 69 @@
```

```
- if (mesh().time().timeIndex() > 1)  
+ if (mesh_.time().timeIndex() > 1)
```

Note: mesh() (an fvModel accessor method) becomes mesh_ (a directly-accessible fv::option member) -- a small base-class naming difference, not a behaviour change.

```
@@ OF-13 line 67 / v2606 line 75 @@
```

```
- bool Foam::fv::flotation::movePoints()  
+  
+ bool Foam::fv::flotationSource::read(const dictionary& dict)  
+ {  
+ if (option::read(dict))
```

Note: OF-13's movePoints() definition sits exactly where v2606's read() definition begins -- see the mesh-motion-hooks note on the next block for why the former isn't needed.

```
@@ OF-13 line 72 / v2606 line 83 @@
```

```
- void Foam::fv::flotation::topoChange(const polyTopoChangeMap& map)  
- {}  
-  
- void Foam::fv::flotation::mapMesh(const polyMeshMap& map)  
- {}  
-  
- void Foam::fv::flotation::distribute(const polyDistributionMap& map)  
- {}  
-  
- bool Foam::fv::flotation::read(const dictionary& dict)  
- {  
- if (fvModel::read(dict))  
- {  
- return true;  
- }  
- else  
- {
```

Why (Mesh-motion hooks not required by fvOption): OF-13's fvModel base class requires every derived model to implement four mesh-topology-change hooks (movePoints(), topoChange(), mapMesh(), distribute()), even if only to return a trivial default, because Foundation's framework calls them unconditionally on any mesh change. ESI's fv::option base class already supplies no-op defaults for the equivalent hooks, so the port omits them -- nothing is lost, since flotation's own fields don't need special handling across mesh changes (every case in this project used a static mesh, so this was never exercised either way).

```
@@ OF-13 line 91 / v2606 line 85 @@
```

```
- }  
+
```

fvModels/flotation/flotation.H → fvOptions/flotationSource/flotationSource.H

12 differing block(s), 70 lines touched (whitespace-only differences excluded)

```
@@ OF-13 line 1 / v2606 line 1 @@
```

```
- #ifndef flotation_H  
- #define flotation_H  
+ /*-----*\
```

```
+ Class
+ Foam::fv::flotationSource
```

Note: OF-13's include guard is replaced by ESI's fuller Doxygen-style file-header block plus its own include guard -- purely a documentation/header-style difference.

@@ OF-13 line 4 / v2606 line 5 @@

```
- #include "fvModel.H"
+ Description
+ Runs the flotation particle-population transport equations (sectional
+ N.<phase>:<section> fields).
+
+ Example usage, in constant/fvOptions:
+ \verbatim
+ flotation
+ {
+ type flotationSource;
+ }
+ \endverbatim
+
+ Parameters are read from constant/flotationProperties (or the file
+ named by the optional "dict" entry).
+
+ SourceFiles
+ flotationSource.C
+
+ \*-----*/
+
+ #ifndef flotationSource_H
+ #define flotationSource_H
+
+ #include "fvOption.H"
```

Note: Class-level documentation, kept short: what the option does and how to select it, without restating the port history or framework comparison already covered in the architecture note above.

@@ OF-13 line 15 / v2606 line 39 @@

```
- Class flotation Declaration
+ Class flotationSource Declaration
```

@@ OF-13 line 18 / v2606 line 42 @@

```
- class flotation
+ class flotationSource
```

@@ OF-13 line 20 / v2606 line 44 @@

```
- public fvModel
+ public option
```

@@ OF-13 line 31 / v2606 line 55 @@

```
- TypeName("flotation");
+ TypeName("flotationSource");
```

@@ OF-13 line 37 / v2606 line 61 @@

```
- flotation
+ flotationSource
```

@@ OF-13 line 41 / v2606 line 65 @@

```
- const fvMesh& mesh,
- const dictionary& dict
+ const dictionary& dict,
+ const fvMesh& mesh
```

*Why: same as **Constructor argument order**, explained at its first occurrence above.*

@@ OF-13 line 44 / v2606 line 68 @@

```
+
+ //- No copy construct
+ flotationSource(const flotationSource&) = delete;
+
+ //- No copy assignment
+ void operator=(const flotationSource&) = delete;
```

Note: Adds explicit deleted copy-constructor/copy-assignment -- a defensive modernisation ESI's own fvOption implementations commonly include; OF-13's original relies on the (in this case, harmless) implicit ones instead.

@@ OF-13 line 47 / v2606 line 77 @@

```
- virtual ~flotation();
```

```
+ virtual ~flotationSource()  
+ {}
```

Note: Destructor moved inline ({} body in the header) instead of OF-13's separate declaration + out-of-line empty definition in the .C file -- purely stylistic.

@@ OF-13 line 54 / v2606 line 85 @@

```
- //- Correct the model  
- virtual void correct();  
+ //- Run the flotation system's solve() once per timestep  
+ virtual void correct(volVectorField& U);
```

Note: The correct() declaration itself, updated to ESI's per-field signature -- see the architecture note above; its doc-comment is kept to one short line.

@@ OF-13 line 57 / v2606 line 88 @@

```
- // Mesh changes  
-  
- //- Update for mesh motion  
- virtual bool movePoints();  
-  
- //- Update topology using the given map  
- virtual void topoChange(const polyTopoChangeMap&);  
-  
- //- Update from another mesh using the given map  
- virtual void mapMesh(const polyMeshMap&);  
-  
- //- Redistribute or update using the given distribution map  
- virtual void distribute(const polyDistributionMap&);
```

*Why: same as **Mesh-motion hooks not required by fvOption**, explained at its first occurrence above.*

flotationFoam

Part 2 — case-level differences between the OF-13 originals and the v2606 ports pushed to edofrederix/flotationFoam, branch v2606

This part covers the seven cases/ directories ported for this push (bubbleColumn, Nguyen, Tan, Brownian, MSFR2D, MSFR3D, shower). Unlike Part 1, this is a summarised view, not a line-by-line diff: for each case, a status table lists every file that changed, was dropped, or is new, and a short write-up explains what changed and why, referencing a shared legend of recurring patterns. Leftover run artifacts (processorN/, log.*, decomposed mesh binaries, Windows zone-identifier files) present in the local OF-13 checkout are excluded as noise, not case content. **Scope note:** only test cases were touched this phase, not the flotationFoam library source itself, and only bubbleColumn was actually validated (built from this project's already-tested reference case); the other six were brought in line with v2606's schemas and had LogMoM/unavailable BCs replaced so they would run, but were not run end-to-end.

167

Identical

69

Changed

55

Not ported

14

New

Recurring patterns (legend)

Each pattern is explained once here and referenced by name in every case below. Three are flagged as uncertain — translated by inspection of ESI's conventions rather than validated by compiling, meshing or running the case.

Unified solver-dispatch application (foamRun/solver → direct binary)

OF-13's controlDict selects the solver two ways in these cases: application foamRun; plus a solver multiphaseEuler; entry, using Foundation's newer unified dispatcher that picks the actual solver code from the solver keyword. ESI v2606 hasn't adopted this dispatcher for this solver family; the case has to name the standalone binary directly, application reactingMultiphaseEulerFoam;, and the solver entry is dropped since it has no meaning there. Alongside it, startFrom latestTime; is changed to startFrom startTime; so a freshly-cloned case starts clean rather than looking for a previous run's time directories that don't exist yet, and writeCompression uncompressed; becomes writeCompression off; — ESI's Switch parser only recognises true/false/on/off/yes/no, not the extra compressed/uncompressed words Foundation added.

Object-name placeholder from m4 templating

OF-13's N.<phase>.m4 template hard-codes object Nc; because the real name (N.water:3, etc.) was only filled in when prep.sh ran m4 once per particle section. Since the v2606 side no longer generates per-section fields this way (see [N-field per-section templating removed](#)), the file is a plain static field and its object entry names the phase-default field it actually is.

momentumTransport.<phase> → turbulenceProperties.<phase>

A pure rename, content-identical otherwise: OF-13 names each phase's turbulence-closure dictionary momentumTransport.<phase>; ESI v2606 calls the same dictionary turbulenceProperties.<phase>. Only

the filename and the internal object entry change; the turbulence model selected inside (laminar, k-epsilon, etc.) and all its coefficients are untouched.

fvModels + fvConstraints merged into fvOptions

OF-13 (Foundation) splits run-time source terms across two separate dictionaries read by two separate frameworks: constant/fvModels for equation source terms (flotationSource, phaseTurbulenceStabilisation) and system/fvConstraints for post-solve field clamps (limitMag, limitTemperature). ESI v2606 uses a single unified fv::option framework instead, read from one file, constant/fvOptions, that both sides carry over unchanged in substance: flotationSource and phaseTurbulenceStabilisation move over with the same type name, and OF-13's limitMag velocity clamp becomes ESI's equivalent fvOption, limitVelocity; limitTemperature already has the same name in both, it just moves from the constraints file into the options file. Nothing about the clamp values or the physics changes, only which file and which framework they're declared under.

phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs

OF-13's phaseProperties keys most interfacial-model blocks by a concatenated phase-pair name (air_water { }, air_dispersedIn_water { }). ESI v2606's basicMultiphaseSystem (which now has to be named explicitly via type basicMultiphaseSystem; itself a new required entry) instead keys every interfacial model as a list of explicit ordered pairs, (air and water) { } for symmetric models or (air in water) { } for models that depend on which phase is dispersed. The port rewrites every interfacial block this way, and also has to add explicit drag, virtualMass, heatTransfer, phaseTransfer, lift, wallLubrication and turbulentDispersion blocks — even the empty ones, () ; — that ESI's schema requires present but that OF-13 either left to file-level defaults or split across separate momentumTransfer/heatTransfer files that no longer exist on the v2606 side.

diameterModel LogMoM → diameterModel constant

OF-13's diameterModel LogMoM drives a population-balance model (log-normal method of moments: LogMoMCoeffs { dMax; dMin; continuousPhase; sources(coalescence{...} breakup{...}); }) that tracks a whole bubble-size distribution and evolves it via coalescence/breakup kernels. LogMoM is this project's own, separately-forked library and is explicitly out of scope for this push (no LogMoM code or options are being pushed to this repository). Every case's diameter model is switched to ESI's standard constant model instead — a single fixed, representative bubble/droplet diameter per phase (constantCoeffs { d ...; }) — removing the LogMoM wrapper and its coalescence/breakup sources along with it.

p_rgh: prghPressure → prghTotalPressure

OF-13's p_rgh outlet/atmosphere boundary uses type prghPressure; with a single p entry. This boundary condition does not exist (or is broken) in this v2606 build; the closest available ESI equivalent is prghTotalPressure, which reconstructs the same total, non-hydrostatic pressure condition but needs a different entry set: p0 instead of p for the reference pressure value, plus explicit U, phi and rho entries naming the phase-specific velocity, flux and density fields (thermo: rho.<phase>) it reconstructs the hydrostatic part from. Every case's p_rgh outlet patch is updated the same way.

N-field per-section m4 templating → single default field

OF-13's particle-size discretisation is section-wise: prep.sh loops over every particle section, using m4 to expand N.<phase>.m4 (and, for particle sections that need it, kappai.<phase>.m4/lambda.<phase>.m4) into one field per section (N.water:1, N.water:2, ...), then runs setLogNormal to fill them from a log-normal size distribution, and records the generated names in bookkeeping files (system/Ncs/Nfs, NcConstraints, NfFieldValues, system/particleVelocities) that fvConstraints/setFieldsDict/post-processing functions then #include. None of this section-wise machinery is part of this push. Every case is simplified to a single default field pair instead — N.<dispersedPhase> initialised to 0, N.<continuousPhase> to a representative number density — relying on flotation's own documented fallback-to-default-field behaviour (already described in the source-level report above) rather than

generating one field per section. The whole generation loop, its bookkeeping files, and everything that `#include` them are removed.

createZonesDict → topoSetDict

OF-13's `createZonesDict` declares cell/face zones with a nested, declarative schema (type `box/cylinder/intersection/union`, an outer `{ ... }` block for boolean combinations, `zoneType cell/face`). ESI v2606 doesn't have this utility/dictionary at all; the equivalent is `topoSetDict`, read by the `topoSet` utility, which instead expresses the same zones as a sequential list of actions — each one a named `cellSet/faceSet/cellZoneSet/faceZoneSet`, built with action `new/add/subset` from a source such as `boxToCell`, `cylinderToCell`, `cylinderToFace`, `patchToFace` or `faceToFace`, finished off with `setToFaceZone/setToCellZone` to turn the working set into the actual zone. Every case using zones is translated this way: the same geometry, expressed as a step-by-step recipe instead of a nested boolean tree. `prep.sh` is updated to run `topoSet` in place of OF-13's `createZones`.

setFieldsDict schema

OF-13's `setFieldsDict` nests named zones under `zones { <name> { type box; zoneType cell; box(...); values{ ... } } }`, on top of a flat `defaultValues { ... }`. ESI's `setFields` utility reads a flatter schema instead: `defaultFieldValues(volScalarFieldValue <field> <value> ...)`; followed by `regions( boxToCell { box(...) ...; fieldValues(...); } ...)`. The port translates every case's region this way; any reference to the removed per-section N-field bookkeeping (e.g. an `#include "NfieldValues"` inside a zone's values) is simply dropped, consistent with the [N-field simplification](#) above.

bubbleColumn's fvSchemes/fvSolution numerics

`bubbleColumn`'s `fvSchemes` (rebuilt from this project's already-validated reference case) needed real corrections beyond the cosmetic header: several `divSchemes` entries that OF-13 expressed as blanket regex wildcards (`"div\(\phi.*,alpha.*\)"`, matching any phase suffix) are spelled out as literal per-phase entries instead (`div(phi,alpha.air)`, `div(phi,alpha.water)`, plus explicit `phir` compression-flux terms), while conversely a couple of entries that were literal in OF-13 become regex on the v2606 side; and the turbulent-diffusion term's density placeholder changes from a bare `rho` to ESI's registered name, `thermo:rho`. `fvSolution` picks up a similar literal/regex adjustment on its residual-control field-name patterns and a new `relaxationFactors` block. **This fix was applied only to `bubbleColumn`.** The other six cases' `fvSchemes/fvSolution` are untouched, byte-identical to their OF-13 originals, and still contain the same wildcard-style entries — whether they match ESI's actual registered field/flux names the way `bubbleColumn`'s now do has not been checked, per the instruction not to validate cases beyond `bubbleColumn`. If any of the other six is run and a term has no matching scheme, this is the first place to look.

Coded fvOption API (MSFR2D / MSFR3D decay-heat source)

OF-13's decay-heat source term is a coded `fvModel`, using that framework's API: the source-adding function is named `codeAddAlphaRhoSup`, the mesh is reached via a `mesh()` accessor function, and the term is added straight into the equation's source via `eqn.source()`. ESI v2606's unified `fv::option` coded template uses a different shape for the same idea: the function is named `codeAddSup`, the mesh is a member reference, `mesh_`, and the option has to name which registered field(s) it applies to up front with a `fields (e.salt)`; entry (the internal-energy field of the salt phase, since this is a phase-specific enthalpy/energy source). The body of the source term itself — reading the prompt and decay heat fields from the `0` directory the first time it's called, then subtracting their volume-weighted sum from the equation's source — is preserved as-is; `eqn.source()` is still the right call inside the coded block in both frameworks, since `fvMatrix::source()` itself hasn't changed. **This translation is a best-effort port by inspection of ESI's coded-fvOption template conventions, not validated by compiling or running the case — flagged here as a high-uncertainty item.**

MSFR3D's injector face zone: nested boolean → sequential topoSet pipeline

MSFR3D's bubbles face zone (the spray-injector geometry) is the most structurally complex zone in any of the seven cases: OF-13 builds it as an intersection of an outer patch-based set (type patch; patch w_core;) with a union of four separate cylinder regions (bubbles1-bubbles4, one per injector). The topoSet translation expresses this as five sequential steps: a faceSet named bubblesAll built by one cylinderToFace new action followed by three add actions (one per remaining injector, unioning them together); a second faceSet named bubbles built by a patchToFace selection of patch w_core; that same bubbles set then narrowed with a subset action using faceToFace against bubblesAll (the intersection step); and finally setToFaceZone to turn the resulting face set into the actual zone. **This is the single highest-uncertainty item in the whole case-level port** — a multi-step set-algebra reconstruction of a nested-boolean OF-13 dictionary, worked out by inspection of ESI's topoSet action vocabulary rather than validated by actually meshing this case.

shower's interfaceCompression block

shower's ported phaseProperties adds an interfaceCompression block, (gas and salt) { type isotropic; cAlpha 1; }, matching the schema every other list-of-pairs interfacial model in this file uses. This is a reasonable best-effort guess at ESI's expected form, not confirmed against a known-working ESI case that uses this particular model — flagged as a secondary uncertainty, lower risk than the MSFR3D zone reconstruction above since a wrong or unrecognised entry here would most likely fail loudly (a missing runtime-selection type) rather than silently run with different physics.

Case: bubbleColumn

Rebuilt directly from this project's already-validated reference case (~/.OpenFOAM/testCases/flotationBubbleColumnTest), not translated field-by-field from the raw OF-13 checkout, and the only case actually run. Its closure-model types were reused as the template for the other six cases wherever their own OF-13 source didn't already specify something different. Beyond the shared patterns below: $p/\alpha/T/U$ initial and boundary values reflect this already-validated setup, not a v2606-forced substitution; `flotationProperties` object entry is corrected to match its filename (`flotationParticleProperties` → `flotationProperties`); and `system/setFieldsDict` is new (bubbleColumn had none in OF-13, relying on `setLogNormal` instead).

8

Identical

23

Changed

11

Not ported

4

New

File	Status	Lines touched
0.org/N.air	Changed	18
0.org/N.water.m4	Not ported (OF-13 only)	-
0.org/T.air	Changed	27
0.org/T.water	Changed	29
0.org/U.air	Changed	18
0.org/U.water	Changed	16
0.org/Vf	Not ported (OF-13 only)	-
0.org/alpha.air	Changed	20
0.org/alpha.water	Changed	20
0.org/alphat.air	Identical	-
0.org/alphat.water	Identical	-
0.org/epsilon.air	Identical	-
0.org/epsilon.water	Identical	-
0.org/k.air	Identical	-
0.org/k.water	Identical	-
0.org/kappai.air.m4	Not ported (OF-13 only)	-
0.org/lambda.air.m4	Not ported (OF-13 only)	-
0.org/nut.air	Identical	-
0.org/nut.water	Identical	-
0.org/p	Changed	11
0.org/p_rgh	Changed	18

clean.sh	Changed	6
constant/flotationProperties	Changed	14
constant/fvModels	Not ported (OF-13 only)	-
constant/g	Changed	12
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/momentumTransport.air → constant/turbulenceProperties.air	Renamed + changed	3
constant/momentumTransport.water → constant/turbulenceProperties.water	Renamed + changed	3
constant/phaseProperties	Changed	177
constant/thermophysicalProperties.air	Changed	12
constant/thermophysicalProperties.water	Changed	12
prep.sh	Changed	34
system/blockMeshDict	Changed	11
system/controlDict	Changed	64
system/decomposeParDict	Changed	12
system/functions	Not ported (OF-13 only)	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Changed	39
system/fvSolution	Changed	24
system/graph	Not ported (OF-13 only)	-
system/particleVelocities	Not ported (OF-13 only)	-
system/particleVelocity.m4	Not ported (OF-13 only)	-
0.org/N.water	New (v2606 only)	-
0.org/Theta	New (v2606 only)	-
constant/fvOptions	New (v2606 only)	-
system/setFieldsDict	New (v2606 only)	-

Patterns: Unified solver-dispatch application (foamRun/solver → direct binary), Object-name placeholder from m4 templating, momentumTransport.<phase> → turbulenceProperties.<phase>, fvModels + fvConstraints merged into fvOptions, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, diameterModel LogMoM → diameterModel constant, p_rgh: prghPressure → prghTotalPressure, N-field per-section m4 templating → single default field, bubbleColumn's fvSchemes/fvSolution numerics

Case: Nguyen

Nitrogen/water bubble column with a "frother" injection zone. Own OF-13 closure choices (drag, lift, wall lubrication) kept where explicitly specified. One leftover OF-13 file, system/createZonesDict (the static output of the removed .m4 template, checked in and never regenerated), was found still sitting in the working tree after the port and has been removed — topoSetDict.m4 fully replaces it.

29

Identical

10

Changed

8

Not ported

2

New

File	Status	Lines touched
0.org/N.nitrogen	Changed	2
0.org/N.water.m4	Not ported (OF-13 only)	-
0.org/T.nitrogen	Identical	-
0.org/T.water	Identical	-
0.org/U.nitrogen	Identical	-
0.org/U.water	Identical	-
0.org/Vf	Not ported (OF-13 only)	-
0.org/alpha.nitrogen	Identical	-
0.org/alpha.water	Identical	-
0.org/alphat.nitrogen	Identical	-
0.org/alphat.water	Identical	-
0.org/epsilon.nitrogen	Identical	-
0.org/epsilon.water	Identical	-
0.org/k.nitrogen	Identical	-
0.org/k.water	Identical	-
0.org/kappai.nitrogen.m4	Not ported (OF-13 only)	-
0.org/lambda.nitrogen.m4	Not ported (OF-13 only)	-
0.org/nut.nitrogen	Identical	-
0.org/nut.water	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Changed	7
bubbleSizeFit.py	Identical	-
clean.sh	Changed	11
constant/flotationProperties	Identical	-

constant/fvModels	Not ported (OF-13 only)	-
constant/g	Identical	-
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/momentumTransport.nitrogen → constant/turbulenceProperties.nitrogen	Renamed + changed	2
constant/momentumTransport.water → constant/turbulenceProperties.water	Renamed + changed	2
constant/phaseProperties	Changed	171
constant/thermophysicalProperties.nitrogen	Identical	-
constant/thermophysicalProperties.water	Identical	-
data/Nguyen-Nyasil-20.txt	Identical	-
data/Nguyen-Snowtex-20L.txt	Identical	-
data/Nguyen-Snowtex-ZL.txt	Identical	-
figStyle.py	Identical	-
plot.py	Identical	-
prep.sh	Changed	34
system/blockMeshDict.m4	Identical	-
system/controlDict	Changed	60
system/createZonesDict	Not ported (OF-13 only)	-
system/createZonesDict.m4 → system/topoSetDict.m4	Renamed + changed	14
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
system/setFieldsDict.m4	Changed	40
0.org/N.water	New (v2606 only)	-
constant/fvOptions	New (v2606 only)	-

Patterns: Object-name placeholder from m4 templating, momentumTransport.<phase> → turbulenceProperties.<phase>, fvModels + fvConstraints merged into fvOptions, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, diameterModel LogMoM → diameterModel constant, p_rgh: prghPressure → prghTotalPressure, N-field per-section m4 templating → single default field, createZonesDict → topoSetDict, setFieldsDict schema, Unified solver-dispatch application (foamRun/solver → direct binary)

Case: Tan

Same nitrogen/water system as Nguyen, same treatment. Where Tan's own OF-13 source was byte-identical to Nguyen's (schema files, scripts), the already-ported Nguyen file was reused directly; where it differed (constant diameter d , wallLubrication set to none on both pairs instead of Nguyen's active Antal-type model), Tan's own OF-13 choice was preserved rather than overridden.

28

Identical

10

Changed

8

Not ported

2

New

File	Status	Lines touched
0.org/N.nitrogen	Changed	2
0.org/N.water.m4	Not ported (OF-13 only)	-
0.org/T.nitrogen	Identical	-
0.org/T.water	Identical	-
0.org/U.nitrogen	Identical	-
0.org/U.water	Identical	-
0.org/Vf	Not ported (OF-13 only)	-
0.org/alpha.nitrogen	Identical	-
0.org/alpha.water	Identical	-
0.org/alphat.nitrogen	Identical	-
0.org/alphat.water	Identical	-
0.org/epsilon.nitrogen	Identical	-
0.org/epsilon.water	Identical	-
0.org/k.nitrogen	Identical	-
0.org/k.water	Identical	-
0.org/kappai.nitrogen.m4	Not ported (OF-13 only)	-
0.org/lambda.nitrogen.m4	Not ported (OF-13 only)	-
0.org/nut.nitrogen	Identical	-
0.org/nut.water	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Changed	7
clean.sh	Changed	11
constant/flotationProperties	Identical	-
constant/fvModels		-

	Not ported (OF-13 only)	
constant/g	Identical	-
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/momentumTransport.nitrogen → constant/turbulenceProperties.nitrogen	Renamed + changed	2
constant/momentumTransport.water → constant/turbulenceProperties.water	Renamed + changed	2
constant/phaseProperties	Changed	171
constant/thermophysicalProperties.nitrogen	Identical	-
constant/thermophysicalProperties.water	Identical	-
data/Tan-00.txt	Identical	-
data/Tan-40.txt	Identical	-
data/Tan-60.txt	Identical	-
figStyle.py	Identical	-
plot.py	Identical	-
prep.sh	Changed	34
system/blockMeshDict.m4	Identical	-
system/controlDict	Changed	60
system/createZonesDict	Not ported (OF-13 only)	-
system/createZonesDict.m4 → system/topoSetDict.m4	Renamed + changed	14
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
system/setFieldsDict.m4	Changed	40
0.org/N.water	New (v2606 only)	-
constant/fvOptions	New (v2606 only)	-

Patterns: Object-name placeholder from m4 templating, momentumTransport.<phase> → turbulenceProperties.<phase>, fvModels + fvConstraints merged into fvOptions, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, diameterModel LogMoM → diameterModel constant, p_rgh: prghPressure → prghTotalPressure, N-field per-section m4 templating → single default field, createZonesDict → topoSetDict, setFieldsDict schema, Unified solver-dispatch application (foamRun/solver → direct binary)

Case: Brownian

Air/water, water always the continuous phase. Its own simpler OF-13 closure set (a single dispersed-only drag/lift/turbulentDispersion pair, TomiyamaCorrelated/Burns, plus its own blending block) was preserved as-is rather than overridden by bubbleColumn's template. turbulenceProperties.* were already named that way in the OF-13 source, so no rename was needed. prep.sh simplifies to essentially `cp -r 0.org 0; runApplication blockMesh` — no zones, no decomposition, nothing left to template once the per-section N-field loop is gone.

23

Identical

6

Changed

10

Not ported

2

New

File	Status	Lines touched
0.org/N.air	Changed	5
0.org/N.water.m4	Not ported (OF-13 only)	-
0.org/T.air	Identical	-
0.org/T.water	Identical	-
0.org/U.air	Identical	-
0.org/U.water	Identical	-
0.org/Vf	Not ported (OF-13 only)	-
0.org/alpha.air	Identical	-
0.org/alpha.water	Identical	-
0.org/alphabet.water	Identical	-
0.org/epsilon.water	Identical	-
0.org/k.water	Identical	-
0.org/kappai.air.m4	Not ported (OF-13 only)	-
0.org/lambda.air.m4	Not ported (OF-13 only)	-
0.org/nut.water	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Changed	7
clean.sh	Changed	6
constant/flotationProperties	Identical	-
constant/fvModels	Not ported (OF-13 only)	-
constant/g	Identical	-
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/phaseProperties	Changed	93
constant/thermophysicalProperties.air	Identical	-
constant/thermophysicalProperties.water	Identical	-

constant/turbulenceProperties.air	Identical	-
constant/turbulenceProperties.water	Identical	-
figStyle.py	Identical	-
plot.py	Identical	-
prep.sh	Changed	35
system/Ncs	Not ported (OF-13 only)	-
system/Nfs	Not ported (OF-13 only)	-
system/blockMeshDict	Identical	-
system/controlDict	Changed	42
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
system/graph	Not ported (OF-13 only)	-
0.org/N.water	New (v2606 only)	-
constant/fvOptions	New (v2606 only)	-

Patterns: Object-name placeholder from m4 templating, fvModels + fvConstraints merged into fvOptions, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, diameterModel LogMoM → diameterModel constant, p_rgh: prghPressure → prghTotalPressure, N-field per-section m4 templating → single default field, Unified solver-dispatch application (foamRun/solver → direct binary)

Case: MSFR2D

Helium/salt, non-isothermal (decay-heat), 2D axisymmetric core+pump geometry. Does not use flotation's N-transport at all, in OF-13 or v2606 — its options only ever contained a heatSource term and turbulence/temperature limiters. (An earlier draft of this port mistakenly added a flotation entry here by over-applying the bubbleColumn template; re-checking the actual OF-13 source caught this before finalising.) No diameterModel/LogMoM in this case either — both phases already used a pure phase model without a size model in OF-13. The separate OF-13 momentumTransfer/heatTransfer files merge into phaseProperties' own heatTransfer block. controlDict drops the vesselMeanMoments function object (it averaged the now-removed moment fields); vesselMeanVoidFraction is kept.

25

Identical

6

Changed

7

Not ported

2

New

File	Status	Lines touched
0.org/T.helium	Identical	-
0.org/T.salt	Identical	-
0.org/U.helium.m4	Identical	-
0.org/U.salt	Identical	-
0.org/alpha.helium	Identical	-
0.org/alpha.salt	Identical	-
0.org/alphat.helium	Identical	-
0.org/alphat.salt	Identical	-
0.org/common	Identical	-
0.org/decayHeatSource	Identical	-
0.org/epsilon.salt	Identical	-
0.org/k.salt	Identical	-
0.org/kappai.helium.m4	Not ported (OF-13 only)	-
0.org/lambda.helium.m4	Not ported (OF-13 only)	-
0.org/nut.salt	Identical	-
0.org/omega.salt	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Identical	-
0.org/promptHeatSource	Identical	-
clean.sh	Changed	6
constant/fvModels	Not ported (OF-13 only)	-

constant/g	Identical	-
constant/heatTransfer	Not ported (OF-13 only)	-
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/momentumTransport.helium → constant/turbulenceProperties.helium	Renamed + changed	2
constant/momentumTransport.salt → constant/turbulenceProperties.salt	Renamed + changed	2
constant/phaseProperties	Changed	117
constant/thermophysicalProperties.helium	Identical	-
constant/thermophysicalProperties.salt	Identical	-
prep.sh	Changed	16
system/blockMeshDict.m4	Identical	-
system/controlDict	Changed	37
system/createZonesDict	Not ported (OF-13 only)	-
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
system/mapFieldsDict	Identical	-
constant/fvOptions	New (v2606 only)	-
system/topoSetDict.m4	New (v2606 only)	-

Patterns: momentumTransport.<phase> → turbulenceProperties.<phase>, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, createZonesDict → topoSetDict, Unified solver-dispatch application (foamRun/solver → direct binary), Coded fvOption API (MSFR2D / MSFR3D decay-heat source)

Case: MSFR3D

Same physical system as MSFR2D, full 3D geometry with a multi-injector spray zone. Treated the same way: not validated, just brought in line with v2606's schemas. createPatchDict is left unchanged — it references the bubbles faceZone by name, unaffected by how the zone is now built.

25

Identical

7

Changed

6

Not ported

1

New

File	Status	Lines touched
0.org/T.helium	Identical	-
0.org/T.salt	Identical	-
0.org/U.helium.m4	Identical	-
0.org/U.salt	Identical	-
0.org/alpha.helium	Identical	-
0.org/alpha.salt	Identical	-
0.org/alphat.helium	Identical	-
0.org/alphat.salt	Identical	-
0.org/common	Identical	-
0.org/decayHeatSource	Identical	-
0.org/epsilon.salt	Identical	-
0.org/k.salt	Identical	-
0.org/kappai.helium.m4	Not ported (OF-13 only)	-
0.org/lambda.helium.m4	Not ported (OF-13 only)	-
0.org/nut.salt	Identical	-
0.org/omega.salt	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Identical	-
0.org/promptHeatSource	Identical	-
clean.sh	Changed	6
constant/fvModels	Not ported (OF-13 only)	-
constant/g	Identical	-
constant/heatTransfer	Not ported (OF-13 only)	-
constant/momentumTransfer	Not ported (OF-13 only)	-

constant/momentumTransport.helium → constant/turbulenceProperties.helium	Renamed + changed	2
constant/momentumTransport.salt → constant/turbulenceProperties.salt	Renamed + changed	2
constant/phaseProperties	Changed	117
constant/thermophysicalProperties.helium	Identical	-
constant/thermophysicalProperties.salt	Identical	-
prep.sh	Changed	13
system/controlDict	Changed	37
system/createPatchDict	Identical	-
system/createZonesDict → system/topoSetDict	Renamed + changed	117
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
system/mapFieldsDict	Identical	-
constant/fvOptions	New (v2606 only)	-

Patterns: momentumTransport.<phase> → turbulenceProperties.<phase>, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, createZonesDict → topoSetDict, MSFR3D's injector face zone: nested boolean → sequential topoSet pipeline, Unified solver-dispatch application (foamRun/solver → direct binary), Coded fvOption API (MSFR2D / MSFR3D decay-heat source)

Case: shower

Gas/salt spray shower, salt is the dispersed droplet phase. Flotation's N-transport was already disabled in the OF-13 source itself (the `flotation fvModel` and `libflotation.so` were commented out); the port preserves that disabled state as a commented-out block with a short explanatory comment, rather than re-enabling anything. `createPatchDict` previously built its patch from its own inline zone spec, independent of the (dead, unused) OF-13 `createZonesDict`; the port instead has it reference the zone `topoSetDict` now builds — the safer, more consistent option now that one exists. `prep.sh` is mostly preserved verbatim (pure Python spray-cone geometry math, version-independent).

29

Identical

7

Changed

5

Not ported

1

New

File	Status	Lines touched
0.org/T.gas	Identical	-
0.org/T.salt	Identical	-
0.org/U.gas	Identical	-
0.org/U.salt	Identical	-
0.org/alpha.gas	Identical	-
0.org/alpha.salt	Identical	-
0.org/alphat.gas	Identical	-
0.org/alphat.salt	Identical	-
0.org/epsilon.gas	Identical	-
0.org/epsilon.salt	Identical	-
0.org/k.gas	Identical	-
0.org/k.salt	Identical	-
0.org/kappai.salt.m4	Not ported (OF-13 only)	-
0.org/lambda.salt.m4	Not ported (OF-13 only)	-
0.org/nut.gas	Identical	-
0.org/nut.salt	Identical	-
0.org/p	Identical	-
0.org/p_rgh	Identical	-
clean.sh	Identical	-
constant/flotationProperties	Identical	-
constant/fvModels	Not ported (OF-13 only)	-
constant/g	Identical	-
constant/momentumTransfer	Not ported (OF-13 only)	-
constant/momentumTransport.gas → constant/turbulenceProperties.gas	Renamed + changed	2
constant/momentumTransport.salt → constant/turbulenceProperties.salt	Renamed + changed	2

constant/phaseProperties	Changed	160
constant/thermophysicalProperties.gas	Identical	-
constant/thermophysicalProperties.salt	Identical	-
makeCones/Make/files	Identical	-
makeCones/Make/options	Identical	-
makeCones/makeCones.C	Identical	-
prep.sh	Changed	14
properties.py	Identical	-
system/blockMeshDict.m4	Identical	-
system/controlDict	Changed	24
system/createPatchDict	Changed	21
system/createZonesDict → system/topoSetDict	Renamed + changed	21
system/decomposeParDict	Identical	-
system/fvConstraints	Not ported (OF-13 only)	-
system/fvSchemes	Identical	-
system/fvSolution	Identical	-
constant/fvOptions	New (v2606 only)	-

Patterns: momentumTransport.<phase> → turbulenceProperties.<phase>, phaseProperties: name-keyed dict-of-dicts → list of ordered phase pairs, shower's interfaceCompression block, createZonesDict → topoSetDict, Unified solver-dispatch application (foamRun/solver → direct binary), N-field per-section m4 templating → single default field

Explicitly out of scope: cases/shower/makeCones/makeCones.C (plus its Make/files/Make/options) is a case-local compiled C++ spray-cone-geometry utility, left completely untouched — case-local but still compiled source, so it falls outside this phase's "test cases, not source" boundary in the strictest reading.

Excluded entirely: MSFR3DMesh

cases/MSFR3DMesh/ is left completely unmodified. It is not a runnable flotationFoam case: it is a standalone mesh-generation utility whose own `build.sh` explicitly pins itself to OpenFOAM/11 — older than the OF-13 baseline this whole report compares against — with a comment noting a known bug in `stitchMesh` between OpenFOAM-11 and 13. It touches none of `phaseProperties`, `fvOptions`, `LogMoM` or boundary conditions; porting its `createZones/rotateMesh/stitchMesh/mergeMeshes` toolchain to v2606 is a separate, higher-risk undertaking deliberately left out of this pass.

Summary of flagged items

Highest uncertainty: MSFR3D's injector face zone ([topoSet reconstruction](#)) — a multi-step set-algebra translation of a nested-boolean OF-13 geometry, not validated by meshing the case.

High uncertainty: MSFR2D/MSFR3D's coded decay-heat [fvOption](#) — translated by inspection of ESI's coded-fvOption API, not validated by compiling or running.

Secondary uncertainty: shower's [interfaceCompression](#) schema — a reasonable guess at ESI's expected form, not confirmed against a known-working ESI case using this model.

Unvalidated fvSchemes/fvSolution: six of the seven cases (everything except bubbleColumn) still carry their OF-13 fvSchemes/fvSolution completely unchanged — see [bubbleColumn's fvSchemes/fvSolution numerics](#) for what bubbleColumn itself needed. This has not been checked for the other six.

Explicit scope exclusions: cases/shower/makeCones/makeCones.C (case-local compiled source) and all of cases/MSFR3DMesh/ (standalone OF-11-pinned meshing utility) — see their sections above.

Cases that never used flotation's N-transport, in OF-13 or v2606: MSFR2D and MSFR3D (non-isothermal decay-heat cases, heatSource only) and shower (already commented out in the OF-13 source). Only bubbleColumn, Nguyen, Tan and Brownian actually exercise the population/attachment physics this port is about.